

AI Assisted Coding Assignment (Lab 15.3)

Sontireddy akshithreddy

2303a51383

BT.NO: 06

****Lab Assignment 15.3 – AI Assisted Coding****

****Task 1: Online Food Ordering API****

****Question****

Design a REST API for an online food ordering system with CRUD operations.

****Prompt (used with AI)****

"Generate a Python Flask REST API for an online food ordering system with endpoints for menu listing, placing orders, updating, tracking, and deleting orders. Include comments and basic validation."

****Code****

```
```python
```

```
from flask import Flask, jsonify, request
```

```
app = Flask(__name__)
```

```
menu = [
 {"id": 1, "name": "Pizza", "price": 250},
 {"id": 2, "name": "Burger", "price": 120}
]
```

```
orders = []
```

```
@app.route('/menu', methods=['GET'])
```

```
def get_menu():
```

```
 return jsonify(menu)
```

```
@app.route('/order', methods=['POST'])
```

```
def place_order():
```

```
 data = request.json
```

```
 order = {"id": len(orders)+1, "items": data["items"], "status": "Placed"}
```

```
 orders.append(order)
```

```
 return jsonify(order)
```

```
@app.route('/order/<int:id>', methods=['GET'])
```

```
def get_order(id):
```

```
 for order in orders:
```

```
 if order["id"] == id:
```

```
 return jsonify(order)
```

```
 return jsonify({"error": "Order not found"}), 404
```

```
@app.route('/order/<int:id>', methods=['PUT'])
```

```
def update_order(id):
```

```
 data = request.json
```

```
 for order in orders:
```

```
 if order["id"] == id:
```

```
 order["items"] = data["items"]
```

```

 return jsonify(order)

 return jsonify({"error": "Order not found"}), 404

@app.route('/order/<int:id>', methods=['DELETE'])
def delete_order(id):
 for order in orders:
 if order["id"] == id:
 orders.remove(order)
 return jsonify({"message": "Order deleted"})
 return jsonify({"error": "Order not found"}), 404

app.run(debug=True)
...

```

### ### \*\*Output\*\*

```

...

GET /menu →
[
 {"id":1,"name":"Pizza","price":250},
 {"id":2,"name":"Burger","price":120}
]

POST /order →
{"id":1,"items":["Pizza"],"status":"Placed"}
...

```

### ### \*\*Code Explanation\*\*

- \* Flask is used to create REST API.
- \* `/menu` returns list of dishes.
- \* `/order` creates a new order.
- \* `/order/<id>` supports GET, PUT, DELETE.
- \* Data stored in in-memory lists.

---

---

### ## \*\*Task 2: Employee Payroll API\*\*

#### ### \*\*Question\*\*

Create an API to manage employees and salaries.

#### ### \*\*Prompt\*\*

"Generate a Flask API for employee payroll management with endpoints for adding employees, updating salary, deleting employees, and listing all employees."

---

#### ### \*\*Code\*\*

```
```python
from flask import Flask, jsonify, request

app = Flask(__name__)
```

```
employees = []
```

```
@app.route('/employees', methods=['GET'])
```

```
def get_employees():
```

```
    return jsonify(employees)
```

```
@app.route('/employees', methods=['POST'])
```

```
def add_employee():
```

```
    data = request.json
```

```
    emp = {
```

```
        "id": len(employees)+1,
```

```
        "name": data["name"],
```

```
        "salary": data["salary"]
```

```
    }
```

```
    employees.append(emp)
```

```
    return jsonify(emp)
```

```
@app.route('/employees/<int:id>/salary', methods=['PUT'])
```

```
def update_salary(id):
```

```
    data = request.json
```

```
    for emp in employees:
```

```
        if emp["id"] == id:
```

```
            emp["salary"] = data["salary"]
```

```
            return jsonify(emp)
```

```
    return jsonify({"error": "Employee not found"}), 404
```

```
@app.route('/employees/<int:id>', methods=['DELETE'])
```

```
def delete_employee(id):
```

```
    for emp in employees:
```

```
        if emp["id"] == id:
```

```
        employees.remove(emp)

        return jsonify({"message": "Employee deleted"})

    return jsonify({"error": "Employee not found"}), 404
```

```
app.run(debug=True)
```

```
'''
```

```
---
```

Output

```
'''
```

POST /employees →

```
{"id":1,"name":"John","salary":50000}
```

PUT /employees/1/salary →

```
{"id":1,"name":"John","salary":60000}
```

```
'''
```

```
---
```

Code Explanation

- * Stores employee data in list.
- * Supports CRUD operations.
- * Salary update handled separately.
- * Includes validation for missing employee.

```
---
```

```
---
```

**Task 3: Student Information REST API**

**Question**

Build a Student Management API.

**Prompt**

"Create a Flask REST API for student management with add, get, update, and delete operations."

**Code**

```
```python
from flask import Flask, jsonify, request

app = Flask(__name__)

students = []

@app.route('/students', methods=['POST'])
def add_student():
 data = request.json
 student = {
 "id": len(students)+1,
 "name": data["name"],
 "course": data["course"]
 }
 students.append(student)
```

```
 return jsonify(student)
```

```
@app.route('/students', methods=['GET'])
```

```
def get_students():
```

```
 return jsonify(students)
```

```
@app.route('/students/<int:id>', methods=['PUT'])
```

```
def update_student(id):
```

```
 data = request.json
```

```
 for s in students:
```

```
 if s["id"] == id:
```

```
 s["name"] = data["name"]
```

```
 return jsonify(s)
```

```
 return jsonify({"error": "Not found"}), 404
```

```
@app.route('/students/<int:id>', methods=['DELETE'])
```

```
def delete_student(id):
```

```
 for s in students:
```

```
 if s["id"] == id:
```

```
 students.remove(s)
```

```
 return jsonify({"message": "Deleted"})
```

```
 return jsonify({"error": "Not found"}), 404
```

```
app.run(debug=True)
```

```
'''
```

```

```

```
Output
```

```
'''
```



POST /students →

```
{"id":1,"name":"Alice","course":"CSE"}
```

GET /students →

```
[{"id":1,"name":"Alice","course":"CSE"}]
```

...

---

### ### **\*\*Code Explanation\*\***

- \* Manages student records.
- \* Uses list as database.
- \* CRUD operations implemented.
- \* Each student has unique ID.

---

---

## ## **\*\*Task 4: Weather Information API\*\***

### ### **\*\*Question\*\***

Integrate external weather API.

### ### **\*\*Prompt\*\***

"Generate Flask API code to fetch current weather using OpenWeatherMap API."

---

### \*\*Code\*\*

```
```python
import requests

from flask import Flask, jsonify

app = Flask(__name__)

API_KEY = "sk-proj-
vMZZoXTmGybZEzckB0RBCrBffi374lYS1gXN9cBuc3ZVd649SBCoRzx_D0i8AcXIB4EOmTs3V6T3BlbkFJffu
tlCXnL37pRSaAdmc6tWO9OHQWusNZCcnzOhsgaybRnLb9SdQiaY5HJNJUmjfiisnOk_T44A"

@app.route('/weather/current/<city>', methods=['GET'])
def get_weather(city):

    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={sk-proj-
vMZZoXTmGybZEzckB0RBCrBffi374lYS1gXN9cBuc3ZVd649SBCoRzx_D0i8AcXIB4EOmTs3V6T3BlbkFJffu
tlCXnL37pRSaAdmc6tWO9OHQWusNZCcnzOhsgaybRnLb9SdQiaY5HJNJUmjfiisnOk_T44A}"

    response = requests.get(url).json()

    return jsonify({

        "temperature": response["main"]["temp"],

        "humidity": response["main"]["humidity"],

        "condition": response["weather"][0]["description"]

    })

app.run(debug=True)
```
```

### \*\*Output\*\*

```
'''
```

```
GET /weather/current/Chennai →
```

```
{
```

```
"temperature": 300,
```

```
"humidity": 70,
```

```
"condition": "clear sky"
```

```
}
```

```
'''
```

```

```

### **### \*\*Code Explanation\*\***

\* Uses requests library to call API.

\* Extracts temperature, humidity, condition.

\* Returns structured JSON response.

```

```

```

```

### **## \*\*Task 5: Delete Item API\*\***

### **### \*\*Question\*\***

Create DELETE endpoint to remove item by index.

### **### \*\*Prompt\*\***

"Write Flask DELETE API to remove item from list using index with validation."

---

**### \*\*Code\*\***

```
```python
from flask import Flask, jsonify

app = Flask(__name__)

items = [{"name": "Notebook", "price": 250}]

@app.route('/items/<int:index>', methods=['DELETE'])
def delete_item(index):
    if index < 0 or index >= len(items):
        return jsonify({"error": "Item not found"}), 404

    removed_item = items.pop(index)
    return jsonify({"message": "Item deleted", "item": removed_item})

@app.route('/items', methods=['GET'])
def get_items():
    return jsonify(items)

app.run(debug=True)
```
```

---

**### \*\*Output\*\***

...

DELETE /items/0 →

{"message": "Item deleted", "item": {"name": "Notebook", "price": 250}}

GET /items →

[]

...

---

### ### \*\*Code Explanation\*\*

- \* Validates index range.
- \* Removes item using pop().
- \* Returns deleted item.
- \* Ensures error handling.

---

---